

¿Por qué necesitamos conocimiento riguroso en IS?

Clase 8 — Unidad 2: Métodos de Investigación en Ingeniería de Software

Investigación en Ingeniería de Software

Ing. Ángel Rea Guamán, Ph.D.



Cuando el software falla sin evidencia

Tres catástrofes reales donde la ausencia de conocimiento riguroso tuvo consecuencias devastadoras.

Therac-25 (1985–1987)

Máquina de radioterapia con bug de condición de carrera → **6 pacientes muertos** por sobredosis de radiación. Decisión de diseño: eliminar hardware de seguridad sin validación empírica.

Boeing 737 MAX (2018–2019)

Sistema MCAS diseñado sin pruebas rigurosas de comportamiento en condiciones reales → **346 muertos** en dos accidentes. Falta de evidencia sistemática en el proceso de certificación.

Knight Capital (2012)

Bug de despliegue de software → pérdida de **USD 440 millones en 45 minutos**. Un error de ingeniería sin validación previa destruyó décadas de capital en menos de una hora.



¿Estas decisiones de ingeniería se basaron en evidencia sistemática o en intuición?

IS está llena de creencias no validadas

"Software engineering is riddled with folklore — beliefs that are widely held but rarely tested." — Robert Glass, 2002

Ejemplos de "folklore" en IS

→ "Los comentarios hacen el código más legible" — creencia extendida, evidencia contradictoria

→ "El pair programming duplica la productividad" — simplificación sin contexto

→ "Las daily stand-ups mejoran la coordinación" — asumido, raramente medido

→ "TDD siempre produce código de mejor calidad" — depende del contexto y equipo

75%

Decisiones sin evidencia

De los desarrolladores toman decisiones técnicas basadas en experiencia personal o en lo que "siempre ha funcionado", sin respaldo empírico.



Consecuencia: Practicamos una ingeniería de suposiciones, no de conocimiento.

Cinco criterios del conocimiento riguroso en IS

Para que el conocimiento en Ingeniería de Software sea verdaderamente riguroso, debe cumplir con los siguientes criterios fundamentales.



Válido

Mide efectivamente lo que dice medir — validez interna y externa garantizadas.



Reproducible

Otro investigador, con los mismos datos y método, llega al mismo resultado.



Generalizable

Las conclusiones aplican más allá del contexto específico del estudio original.



Revisado por pares

La comunidad científica ha evaluado y validado el proceso metodológico.



Basado en evidencia

Sustentado en datos reales, no en opinión o autoridad personal.



Analogía NASA: Antes de lanzar una misión, la NASA valida cada componente con miles de horas de pruebas documentadas. ¿Por qué el software debería ser diferente?

Sin rigor, no podemos distinguir verdad de falsedad

Fuente: Hussain & Soomro (2023), *Applied Computer Systems*, vol. 28, no. 1 — estudio exploratorio peer-reviewed sobre desinformación digital.

4.59B

Usuarios en redes sociales

En 2022, con proyección de 5.85B para 2027.

10%

Tweets con misinformación

Del análisis de 30.7 millones de tweets en las elecciones EE.UU. 2016.

❌ Artículos con misinformación sobre cáncer recibían **más engagement** que los verídicos — evidencia empírica de salud digital.

Taxonomía validada por investigación rigurosa

Misinformación

Información falsa, **sin intención de daño**.
Error no deliberado.

Desinformación

Información falsa, **con intención de daño**.
Manipulación deliberada.

Malinformación

Información verdadera, **usada para dañar**.
Contexto malicioso.

📖 Lección para IS: Sin métodos rigurosos, no podemos construir sistemas que detecten, filtren ni gestionen información de manera confiable.

La solución: IS basada en evidencia

Origen: Kitchenham et al. (2004) proponen aplicar el modelo de la **medicina basada en evidencia** a la Ingeniería de Software — el marco EBSE.

Formular pregunta

Evaluar evidencia

Evaluar impacto

Buscar evidencia

Aplicar resultados

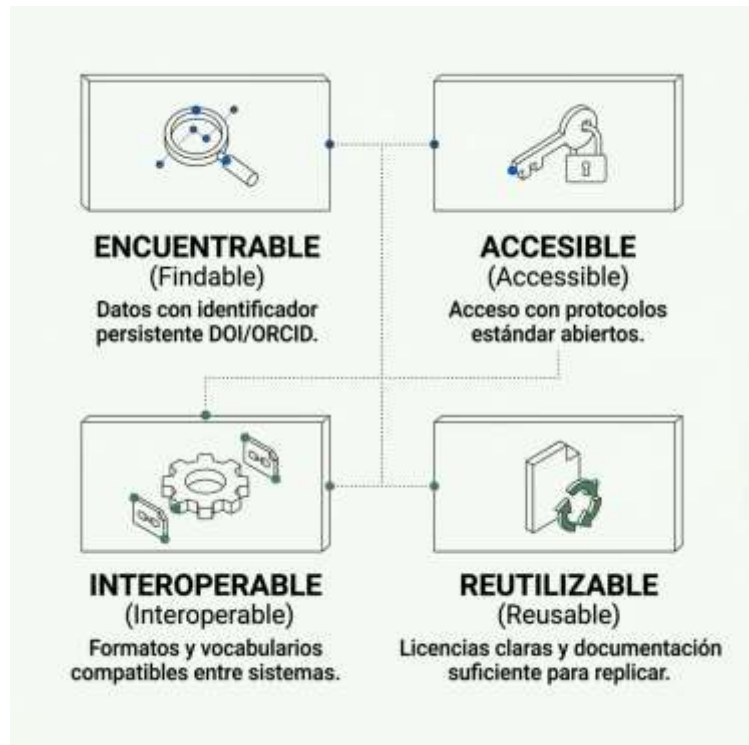
El ciclo EBSE transforma la práctica de IS: cada decisión de diseño pasa por un proceso sistemático de búsqueda, evaluación y aplicación de evidencia — igual que en la medicina clínica moderna.

🌐 **Conexión NASA — Open Science:** La NASA aplica exactamente este ciclo. Cada misión genera datos abiertos, revisados por pares y reproducibles — disponibles para que cualquier investigador valide o refute los hallazgos. Resultado: decisiones de diseño respaldadas por datos, no por intuición.



Sin datos FAIR, el conocimiento queda atrapado

Principios FAIR — Wilkinson et al. (2016): el estándar internacional de rigor para datos de investigación científica y de ingeniería.



Impacto en IS cuando los datos NO son FAIR

No replicable

No se puede replicar el estudio → no sabemos si el resultado es real o un artefacto del contexto.

No acumulable

No se puede comparar con otros estudios → no acumulamos conocimiento colectivo en la disciplina.

Conocimiento efímero

El "conocimiento" generado muere con el paper — no trasciende ni transforma la práctica.

❗ Caso NASA: Los datos de GeneLab, JunoCam y el Telescopio James Webb son públicos, con DOI, en formatos abiertos — cualquier equipo del mundo puede verificarlos.

De artesanía a ciencia: la evolución del rigor en IS

La Ingeniería de Software tardó décadas en reconocer que necesita su propia ciencia rigurosa. Hoy ese reconocimiento es el núcleo de la disciplina.



¿Qué creemos vs. qué sabemos en IS?

Un ejercicio de reflexión crítica: contrastando el folklore extendido con lo que la investigación empírica realmente nos dice.

Creencia común ("folklore")

"Más horas de trabajo = mejor software"

"El pair programming es una pérdida de tiempo"

"Los comentarios mejoran la comprensión del código"

"Ágil siempre es mejor que cascada"

"TDD siempre produce código de mejor calidad"

¿Qué dice la evidencia?

Estudio NASA: productividad cae después de 50h/semana; los errores aumentan significativamente

Williams et al. (2000): mejora la calidad y reduce defectos en un **15%**

Ifflaender et al. (2021): los comentarios desactualizados confunden más de lo que ayudan

La evidencia muestra que depende del contexto, tamaño del equipo y tipo de producto

Resultados mixtos: beneficios dependen de la experiencia del equipo y la complejidad del dominio

❓ Reflexión para el aula: ¿Cuántas de sus decisiones técnicas actuales están basadas en folklore no validado?

Síntesis y próximos pasos

Folklore en IS

IS está llena de creencias no validadas que pueden tener consecuencias graves en sistemas reales.

Conocimiento riguroso

Requiere: validez, reproducibilidad, generalización, revisión por pares y evidencia empírica.

Marco EBSE

Evidence-Based Software Engineering transforma IS en una disciplina científica acumulable.

Principios FAIR

Garantizan que el conocimiento generado sea reproducible, comparable y acumulable.

INVB5 — Investigación Bibliográfica

Identifica **UN mito o creencia común en IS** (ej: "pair programming mejora la productividad", "TDD reduce bugs", "stand-ups diarios mejoran la coordinación"). Busca en **IEEE Xplore o ACM Digital Library** UN artículo científico que lo valide o refute. Responde: ¿qué tipo de estudio usaron?, ¿cuál fue su conclusión principal?, ¿consideras que la evidencia es suficiente para cambiar la práctica?

→ PRÓXIMA CLASE: TIPOS DE MÉTODOS DE INVESTIGACIÓN EN IS